

22-ma'ruza. Metodning massivli parametrlari va params kalit so'zi

Reja:

1. Parametrlar
2. Params kalit so'zi
3. Parametr sifatida massivlardan foydalanish

Metod tanasida bir nechta qiymatlarni qaytarish mumkin. Qanday qilib bitta emas, ikkita qiymatni qaytarish mumkin? Agar qiymatlar bir xil turda bo'lsa, ularni massiv sifatida qaytarish mumkin, agar ular har xil turda bo'lsa, strukturadan foydalanish mumkin. Ammo undan ham yaxshiroq yo'l bor - bitta parametrni qaytarish qiymati sifatida va boshqa parametrni parametrlardan biri orqali qaytarish mumkin.

C# o'zgaruvchilarni aniq turdagi yoki yashirin tur sifatida e'lon qilish imkonini beradi. Aniq turdagi o'zgaruvchilar xuddi e'lon qilingan tur kabi kiritish talab qilinadi, yashirin tur bu - kompilyator ularda saqlangan qiymatga qarab ishlash vaqtida turni aniqlaydi. C# da **var** kalit so'zi yashirin turdagi o'zgaruvchilarni e'lon qilish uchun ishlatiladi.

Agar kodning bir qismi ikki yoki undan ortiq marta takrorlansa, ko'rsatmalarni alohida metodga o'tkazish mantiqan to'g'ri bo'ladi. Klaviaturadan massiv elementlarini kiritish misolini ko'rib chiqamiz:

```
var a = new int[3];
var b = new int[3];
for (int i = 0; i < a.Length; i++)
{
    Console.Write("a[i] = ");
    a[i] = Convert.ToInt32(Console.ReadLine());
}
for (int i = 0; i < b.Length; i++)
{
    Console.Write("b[i] = ");
    b[i] = Convert.ToInt32(Console.ReadLine());
}
```

Ko'rib turganingizdek, massivlarni konsolda kiritish takrorlanadi, shuning uchun biz klaviaturadan kiritilgan qiymatlar bilan massivga qiymat kiritish metodini yaratishimiz mumkin:

```
static int[] Massiv(string MassivNomi, int ElementSoni)
{
    var a = new int[ElementSoni];
    for (int i = 0; i < a.Length; i++)
    {
        Console.Write("{0}[{1}]=", MassivNomi, i);
        a[i] = Convert.ToInt32(Console.ReadLine());
    }
    return a;
}
```

Argumentlar metodga uzatiladi: massiv nomi va elementlar soni (uzunlik).

Shundan so'ng, metod butun sonlarning yangi massivlarini yaratish uchun qayta-qayta ishlatilishi mumkin:

```
public static void Main(string[] args)
{
    int[] t = Massiv("t", 5);
    var g = Massiv("g", 4);
    Console.ReadKey(true);
}
```

Oldingi barcha misollarda doimiy sonli parametrlardan foydalanganmiz. Ammo *params* kalit soʻzidan foydalanib, biz cheksiz miqdordagi parametrlarni metodlarda ishlatishimiz mumkin:

Parametrlar

Koʻpincha metodlarda cheklangan miqdordagi argumentlardan foydalaniladi. Agar dasturdagi metodda argumentlar soni koʻpaysa, kodning koʻrinishini qiyinlashadi. *params* kalit soʻzi orqali cheksiz argumentlarni massiv koʻrinishida kiritish va kodni ancha ixchamlashtirish mumkin. *params* parametrli metodlarni chaqirganda, quyidagilarni kiritish mumkin:

Massiv elementlari turiga oid argumentlarning vergul bilan ajratilgan roʻyxati:

```
int yigindi = Sum(19, 9, 7, 17, 39, 47);
```

Belgilangan turdagi argumentlar massivi:

```
object[] obj = new object[] { "hamroliyev", 'a', 19, 9, 12.8 };
```

```
//Add metodini chaqiramiz
```

```
Add(obj);
```

Hech qanday argument yuborilmasa, *params* roʻyxati uzunligi nolga teng boʻladi. *params* kalit soʻzi dasturchi metodda foydalaniladigan parametrlar soni haqida oldindan maʼlumotga ega boʻlmagan hollarda foydali boʻladi. Metod konstruktorida *params* kalit soʻzidan faqat bir marta foydalanishimiz mumkin. *params* ga tegishli argumentdan keyin qoʻshimcha argument berish mumkin emas.

params kalit soʻzidan foydalanishga oddiy misol:

```
using System;
```

```
namespace params_key
```

```
{
```

```
    class Program
```

```
    {
```

```
        // params parametrni oʻz ichiga olgan metod
```

```
        public static int Sum(params int[] sonlarRoxati)
```

```
        {
```

```
            int sum = 0;
```

```
            foreach (int i in sonlarRoxati)
```

```
            {
```

```
                sum += i;
```

```
            }
```

```
            return sum;
```

```
        }
```

```
        static void Main(string[] args)
```

```
        {
```

```
            //Sum metodini chaqiramiz.
```

```
            int yigindi = Sum(9, 9, 7, 7, 9, 7);
```

```
            Console.WriteLine("Yigʻindi : {0}",yigindi);
```

```
            Console.ReadKey();
```

```
        }
```

```
    }
```

```
}
```

Output: yigindi : 48

Sizda "*params*" kalit soʻzidan foydalanmasdan ham metodlarda massivdan foydalanish mumkinku, *params* kalit soʻzini nima keragi bor?" degan savol tugʻilishi mumkin. *params* kalit soʻzi metoddan foydalanishni ancha qulaylashtiradi: Quyidagi misolda koʻrib chiqamiz:

```
using System;
```

```
namespace params_key
```

```
{
```

```
    class Program
```

```
    {
```

```

// params parametrni o'z ichiga olgan metod
public static int Sum(params int[] sonlarRoyxati)
{
    int sum = 0;
    foreach (int i in sonlarRoyxati)
    {
        sum += i;
    }
    return sum;
}
static void Main(string[] args)
{
    int[] a={19, 9, 7, 17, 39, 47};
    int yigindi = Sum(a);
    // params ishlatmasdan ham metodni bu usulda chaqirish mumkin edi
    Console.WriteLine("yig'indi : {0}", yigindi);
    // paramsdan foydalangandagina metodni shunday chaqirish mumkin:
    Console.WriteLine("yig'indi : {0}", Sum(19, 9, 7, 17, 39, 47));
    // Agar paramsdan foydalanmagan bo'lsangiz, kodning bu qismida xatolik yuzaga keladi,
    //paramsdan foydalansangiz, metodni shu holatda ham chaqirish mumkin
    Console.WriteLine(Sum());
    Console.ReadKey();
}
}
}

```

Metodni belgilashda parametrning o'zi *params* kalit so'zi bilan berilganlar turidagi bir o'lchovli massivni ko'rsatishi kerak. *params* modifikatori bilan parametr o'rniga metodni chaqirganda, individual qiymatlarni ham, qiymatlar satrini ham o'tkazish mumkin yoki umuman parametrlarni o'tkaza olmasligimiz mumkin. Metodga o'tkazilgan qiymatlar soni aniqlanmagan, ammo bu qiymatlarning barchasi *params* bilan parametr turiga mos kelishi kerak.

Agar boshqa parametrlarni o'tkazish kerak bo'lsa, ular *params* kalit so'zi bilan parametrdan oldin ko'rsatilishi kerak:

```

using System;
namespace Massivlar
{
    class Program
    {
        public static void Main(string[] args)
        {
            int[] nums = { 1, 2, 3, 4, 5};
            Sum(10, nums); // 10 soni initialValue parametriga o'tkaziladi
            Sum(1, 2, 3, 4);
            Sum(1, 2, 3);
            Sum(20);
            Console.ReadKey(true);
        }
        public static void Sum(int initialValue, params int[] numbers)
        {
            int result = initialValue;
            foreach (var n in numbers)
            {
                result += n;
            }
        }
    }
}

```

```

    }
    Console.WriteLine(result);
}
}

```

Bu yerda, Sum() metodida zarur bo'lgan initialValue parametri mavjud, shuning uchun metodni chaqirganda, unga qiymat berish kerak. Shuning uchun metod chaqirilganda birinchi qiymat ushbu parametrga o'tkaziladi.

Biroq parametrlarni o'zgartiruvchi parametrdan so'ng, boshqa parametrlarni ko'rsata olmaymiz. Ya'ni quyidagi metod ta'rifi noto'g'ri:

```

public static void Sum(params int[] numbers, int initialValue)
{
    // metod tanasi
}

```

Parametr sifatida massiv

Bundan tashqari, parametrlarni uzatishning bu metodi massivlarni parametr sifatida o'tkazishdan ajralib turishi kerak:

```

using System;
namespace Massivlar
{
    class Program
    {
        public static void Main(string[] args)
        {
            int[] nums = { 1, 2, 3, 4, 5};
            Sum(nums, 10);
            // Sum(1, 2, 3, 4); // bu mumkin emas, massivni o'tkazish kerak
            Console.ReadKey(true);
        }
        public static void Sum(int[] numbers, int initialValue)
        {
            int result = initialValue;
            foreach (var n in numbers)
            {
                result += n;
            }
            Console.WriteLine(result);
        }
    }
}

```

Sum metodi *params* kalit so'zi bo'lmagan massivni parametr sifatida qabul qilganligi sababli, uni chaqirganda, birinchi parametr sifatida massivni metodga o'tkazish kerak. Bundan tashqari, *params* parametrli metoddan farqli o'laroq, massiv parametridan keyin boshqa parametrlarni qo'yish mumkin.

Qiymatlar qatoridan eng kichik qiymatni topadigan metodni yarataylik. Bunday metodga kamida ikki, uch, to'rt yoki undan ko'p qiymat o'tkazilishi mumkin. Lekin har qanday holatda, metod ushbu qiymatlarning eng kichigini qaytarishi kerak. Bunday metodni oddiy parametrlar yordamida yaratish mumkin emas.

Quyida berilgan qiymatlar to'plamining eng kichigini qaytaradigan MinVal() metodini yaratish uchun *params* modifikatoridan foydalanadigan dastur keltirilgan.

```

using System;
class ParamsDemo

```

```

{
    public static void Main()
    {
        int min;
        int a = 10, b = 20;
        // Ikki qiymatli metodni chaqirish
        min = MinVal(a, b);
        Console.WriteLine("Eng kichik qiymat " + min);
        // Uch qiymatli metodni chaqirish
        min = MinVal(a, b, -1);
        Console.WriteLine("Eng kichik qiymat " + min);
        // Besh qiymatli metodni chaqirish
        min = MinVal(18, 23, 3, 14, 25);
        Console.WriteLine("Eng kichik qiymat " + min);
        // Ko'p qiymatli metodni chaqirish
        int[] args = { 45, 67, 34, 9, 112, 8 };
        min = MinVal(args);
        Console.WriteLine("Eng kichik qiymat " + min);
        Console.ReadKey();
    }
    public static int MinVal(params int[] nums)
    {
        int m;
        if (nums.Length == 0)
        {
            Console.WriteLine("Xato: argumentlar yo'q.");
            return 0;
        }
        m = nums[0];
        for (int i = 1; i < nums.Length; i++)
            if (nums[i] < m) m = nums[i];
        return m;
    }
}

```

Dastur bajarilishi natijasida quyidagi qiymatni qaytaradi

Eng kichik qiymat: 10

Eng kichik qiymat: -1

Eng kichik qiymat: 3

Eng kichik qiymat: 8

Har safar MinVal() metodi chaqirilganda, metodga nums massividagi argumentlar uzatiladi. Ushbu massivning uzunligi uzatilgan argumentlar soniga teng. Shuning uchun MinVal() metodidan foydalanib, istalgan qiymatlar sonidan eng kichigini topish mumkin.

MinVal() metodiga oxirgi chaqirishga e'tibor bering. Bir qator qiymatlarni o'z ichiga olgan massiv uzatiladi. *params* turidagi parametr yaratilganda, u o'zgaruvchilar uzunligidagi argumentlar ro'yxatini yoki argumentlarni o'z ichiga olgan massivni qabul qiladi.

params turi parametriga argumentlarning istalgan soni o'tkazilishi mumkin bo'lsa-da, ularning barchasi ushbu parametr tomonidan belgilangan massiv turiga ega bo'lishi kerak.

Masalan, MinVal() metodini chaqirish

```
int min = MinVal(1, 2.2); // Noto'g'ri!
```

params modifikatoridan chegara shartlariga rioya qilgan holda foydalanish kerak, chunki *params* tipidagi parametr istalgan miqdordagi argumentlarni qabul qilishi mumkin, hatto nolga teng bo'lsa ham. Misol uchun, quyidagi kodda MinVal() metodini chaqirish C# sintaksisi nuqtayi nazaridan to'g'ri hisoblanadi.

```
min = MinVal(); // argument yo‘q
```

```
min = MinVal(3); // 1 ta argument
```

Shuning uchun `MinVal()` metodi ushbu elementga kirishdan oldin `nums` massivida kamida bitta element mavjudligini tekshiradi. Bunday tekshiruvsiz `MinVal()` metodini argumentlarsiz chaqirish istisnosiga olib keladi.

Metod ham oddiy parametrlarga, ham o‘zgaruvchan uzunlikdagi parametrga ega bo‘lishi mumkin. Masalan, quyida `ShowArgs()` metodi bitta string tipidagi parametрни va *params* turidagi parametr sifatida butun son massivini oladigan dastur keltirilgan.

```
using System;
```

```
namespace parametr
```

```
{
```

```
class MyClass
```

```
{
```

```
    public void ShowArgs(string xabar, params int[] nums)
```

```
    {
```

```
        Console.Write(xabar + " ");
```

```
        foreach (int i in nums)
```

```
            Console.Write(i + " ");
```

```
        Console.WriteLine();
```

```
    }
```

```
}
```

```
class ParamsDemo2
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        MyClass ob = new MyClass();
```

```
        ob.ShowArgs("Bu butun sonlar qatoridir: ", 1, 2, 3, 4, 5);
```

```
        ob.ShowArgs("Bu esa ikkita butun son", 17, 20);
```

```
        Console.ReadKey();
```

```
    }
```

```
}
```

```
}
```

Bu dasturni ishga tushirish natijasi.

Bu butun sonlar qatoridir: 1 2 3 4 5

Bu esa ikkita butun son 17 20

Metod oddiy string turidagi parametrga, hamda *params* turidagi parametrga ega bo‘lgan hollarda, u ushbu metod uchun parametrlar ro‘yxatida oxirida ko‘rsatilgan. Lekin har qanday holatda, *params* turi parametri yagona bo‘lishi kerak.